

1) Definición del objetivo

- Convertir el formulario original (temática de autos) en un **formulario gamer** para **ingresar videojuegos a inventario**.
- **Campos iniciales (5):** título, plataforma, género, precio, cantidad.
- **Requisitos UX:** tema oscuro neon, validación en tiempo real, vista previa y confirmación visual de guardado.
- **Persistencia:** almacenar cada alta en localStorage.

2) Estructura base del HTML

- Se creó index.html con:
 - <head>: fuentes (Press Start 2P + Inter) y estilos.
 - <body>: contenedor .container, cabecera .header (logo + título + toast) y formulario.
 - <form id="gameForm" novalidate> con **grid** de 2 columnas.
- Se dejó un <div id="preview"> para la **vista previa**.

Decisión: usar novalidate para controlar los mensajes con JS personalizado.

3) Diseño visual “neón gamer”

- Paleta y variables CSS: --primary (morado), --primary-2 (cian), sombras y --ring para foco.
- Caja contenedora con **glass/gradient** sutil y sombras.
- Inputs con estados .valid y .invalid y .error.show para mensajes.

4) Campos del formulario (v1 con 5 campos)

1. **Título** (text)
2. **Plataforma** (select) – opciones: PS5, Xbox Series, Switch, PC, Steam Deck
3. **Género** (select) – acción, aventura, RPG, etc.
4. **Precio** (number, step="0.01", min="0")
5. **Cantidad** (number, entero ≥0)

Cada campo quedó con su .

5) Script principal y patrón de helpers

- Archivo gamer-stock.js con helper:
- `const $ = (id) => document.getElementById(id);`
- `function setValid(el, ok, msgId) { /* agrega/quita .valid/.invalid y .error.show */ }`
- **Reglas** de validación centralizadas en un objeto:
- `const reglas = {`
- `titulo: (v) => v.trim().length >= 3,`
- `plataforma: (v) => !!v,`
- `genero: (v) => !!v,`
- `precio: (v) => Number(v) > 0 && Number(v) <= 9999.99,`
- `cantidad: (v) => Number.isInteger(Number(v)) && Number(v) >= 0 && Number(v) <= 9999,`
- `};`

6) Validación en tiempo real (v1)

- input/change por campo:
 - **Título:** longitud mínima.
 - **Plataforma/Género:** seleccionado.
 - **Precio:** patrón de **máx 2 decimales**; se corrige a `toFixed(2)` si se excede.
 - **Cantidad:** se fuerza a **solo dígitos** con `replace(/\D+/g, "")`.

7) Vista previa reactiva

- Función `renderPreview(data)` que arma el resumen con emojis (🎮, 💻, 🌱, 📦, 📦).
- Se llama cada que cualquier input cambia.
- `function renderPreview({ titulo, plataforma, genero, precio, cantidad }) { /* ... */ }`

8) Guardado en localStorage

- Clave STORAGE_KEY = 'stock_gamer_items'.
- saveItem(alta) agrega el nuevo ítem al arreglo persistido.
- Se genera id: crypto.randomUUID() para cada alta.

9) Envío del formulario (submit)

- preventDefault().
- Se validan todas las reglas **otra vez** (seguridad).
- Si hay error, se hace focus() en el primer .invalid.
- Si todo OK:
 - Se construye el objeto alta.
 - Se guarda con saveItem(alta).
 - Se muestra **toast** “Guardado en inventario ”.
 - form.reset() y limpieza de estados .valid/.invalid y .error.

10) Botón “Limpiar”

- type="reset" más un listener que:
 - limpia clases de estado,
 - oculta errores,
 - reinicia la vista previa.

11) Incorporación del campo Fecha (v2)

- Se añadió el **6º campo**:
- <input type="date" id="fecha" name="fecha" required>
- **Restricción** igual al ejemplo original: **no permitir fechas pasadas**.
 - En lógica:
 - fecha: (v) => {
 - if (!v) return false;
 - const sel = new Date(v);
 - const hoy = new Date(); hoy.setHours(0,0,0,0);

- `return sel >= hoy;`
- `}`
- En UI: se fija min dinámico a hoy:
- `function isoLocalDateToday(){ /* arma YYYY-MM-DD local */ }`
- `inputs.fecha.setAttribute('min', isoLocalDateToday());`
- Se valida al `change` y se agregó a `renderPreview()`.

12) Accesibilidad y UX sutiles

- `aria-live="polite"` en la vista previa para lectores de pantalla.
- Foco automático en primer campo inválido al enviar.
- `:focus` con **anillo** visible (no solo color del borde).

13) Seguridad/robustez

- Conversión numérica con `Number()` y chequeos `Number.isFinite`.
- `try/catch` al leer `localStorage`.
- Normalización de fecha a medianoche **local** para evitar falsos negativos.

14) Pruebas recomendadas

- **Título:** “Go” (debe marcar error), “GOW” (válido).
- **Plataforma/Género:** enviar sin seleccionar → error.
- **Precio:** 0, -5, 100000 → error; 9999.99 OK; 12.345 se ajusta o marca.
- **Cantidad:** -1, 10.5, abc → error; 0 y 9999 OK.
- **Fecha:** ayer → error; hoy/mañana → OK.
- **Reset:** debe limpiar estados y preview.
- Persistencia: enviar 2–3 altas y revisar `localStorage` (DevTools).

15) Estructura final de archivos

/tu-proyecto

└─ index.html (interfaz, estilos embebidos)

└─ gamer-stock.js (validación, preview, storage, fecha)

16) Cómo ejecutarlo

1. Guarda ambos archivos en la misma carpeta.
 2. Abre index.html en tu navegador (doble clic).
 3. Abre la **Consola** del navegador para ver las altas guardadas (logs) y en **Application** → **Local Storage** verás stock_gamer_items.
-

Cambios clave entre versiones

- **v1 (5 campos):** título, plataforma, género, precio, cantidad; validación + preview + storage.
- **v2 (+Fecha):** se añadió **Fecha de ingreso** con:
 - Validación “≥ hoy”.
 - min dinámico en el <input type="date">.
 - Integración en preview y objeto guardado.